Measuring a Tiny Vibration from a Flying Drone's Video

How we tracked a cable moving less than one and a half pixels

Quoc-Bao Ta[1] Anh Le[2] Trung Ha Nguyen[3] Thai Ha Dang[4]

[1]Department of Civil Engineering, University of Texas Rio Grande Valley, Edinburg, TX, USA
[2]Department of Civil, Construction and Environmental Engineering, North Dakota State University, Fargo, ND, USA
[3]AI & Robotics Research Engineer, Jisan Construction Co., Ltd., Busan, South Korea
[4]Department of Electrical Engineering, University of North Texas, Denton, TX, USA

SHM Competition — Project 1: UAV-based vision displacement measurement

August 30, 2026

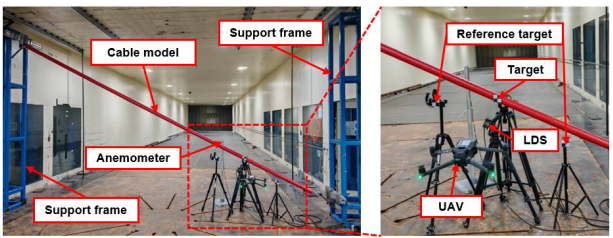
The story in one page

1. **The challenge:** a cable in a wind tunnel shakes by about 1 mm. A drone films it. Can we measure that shake from the video alone?
2. **What people tried before:** template matching, optical flow, deep-learning trackers. Each is good at something, but each has a weak spot here.
3. **Our backbone:** "local phase" — a way to read motion from the *timing* of wave patterns in the picture.
4. **Our changes:** three fixes so that local phase works on a shaky drone video.
5. **The experiment:** compare against a laser sensor. Result: error of 0.075 mm.
6. **What we learned:** the tracker is no longer the problem — the drone's own movement is.

Part 1: The challenge

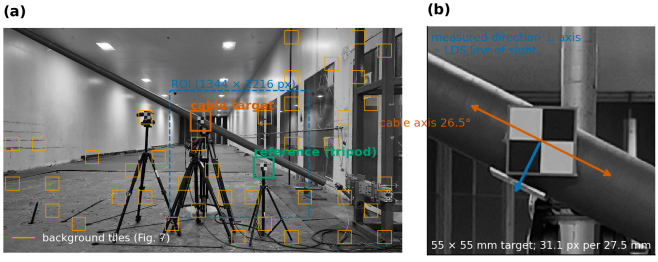
1. Part 1 — The Challenge
2. Part 2 — What people tried before
3. Part 3 — The backbone: local phase
4. Part 4 — Our adaptations (and the hypotheses behind them)
5. Part 5 — Experiments and results
6. Part 6 — Discussion and conclusion

The setup: a cable, a wind tunnel, a drone, and a laser



- A long **cable model** is tilted at 27° inside a wind tunnel.
- Wind at 3.6 m/s makes the cable **vibrate** (like a guitar string humming in a breeze).
- A small **checkerboard target** (55 mm × 55 mm) is glued to the cable.
- A **drone** hovers about 2 m away and films the target.
- A **laser sensor (LDS)** measures the true motion. We use it only to *check* our answer.

What exactly do we have to measure?



- The video is 4K (3840 × 2160 pixels), 50 pictures per second, 60 seconds long.
- We must output **one number per frame**: how far the target has moved, in millimetres, *perpendicular* to the cable (the direction the laser looks).
- There is a **second checkerboard on a tripod**. It does not move. We will need it soon!

Key idea: Input = video. Output = a curve of displacement vs. time at 50 Hz.

Why is this hard? Three reasons

1. **The motion is tiny.**
   The checkerboard is only 31 pixels wide. The vibration is about ± 1.5 pixels. One pixel is 0.88 mm. So we need accuracy *much better than one pixel* — "sub-pixel" accuracy.

2. **The camera is moving too.**
   A hovering drone never stays perfectly still. Over the minute it drifts by about ± 18 pixels — **ten times bigger** than the signal we want.

3. **The video is compressed.**
   The file is H.264 (like YouTube). Compression adds small "noise" to every picture, which blurs the exact edge positions.

Think about it: If someone whispers 1 mm of motion while a drone "shouts" 18 mm of motion, how would you hear the whisper?

How is the answer scored?

- The laser sensor gives the "true" motion at 10,000 samples per second.
- We compare our curve to the laser curve and compute the **RMSE** (root-mean-square error):

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{t}\left(\mathrm{ours}(t) - \mathrm{laser}(t)\right)^2}$$

  In plain words: the "typical" distance between our curve and the true curve. Smaller is better.
- Two small traps in the data notes:
  - The laser and the video clocks start at slightly different times (less than 1 s). We must **align** them first (we slide one curve until it best matches the other).
  - The laser is not zero at the start, so we **subtract the average** of each curve before comparing.
- For scale: the vibration itself has a spread of 0.38 mm. An error of 0.1 mm would already be pretty good.

Part 2: What people tried before

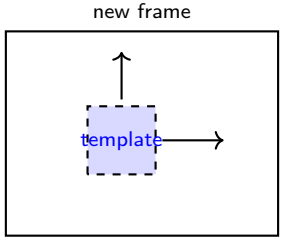
1. Part 1 — The Challenge
2. Part 2 — What people tried before
3. Part 3 — The backbone: local phase
4. Part 4 — Our adaptations (and the hypotheses behind them)
5. Part 5 — Experiments and results
6. Part 6 — Discussion and conclusion

Method A: Template matching (ZNCC)

Idea: cut out a small picture of the target from frame 0 (the "template"). In every later frame, slide the template around and find where it fits best.

- "ZNCC" is just the score used for "fits best" — it ignores brightness changes.
- The best fit lands on a whole pixel. To go below one pixel, we fit a small curve (parabola) to the scores around the peak and read its top.

What it is good at: simple, robust, never loses the target (minimum match score 0.977 over the whole video).
Weak spot here: the parabola trick is only an approximation. On compressed video its error is about 0.04–0.07 px — and we need better.



slide it everywhere, keep the best score

Method B: Deep-learning point trackers (CoTracker3)

Idea: a neural network trained on millions of videos learns to follow any point you click on, frame after frame.

- **Good at:** long videos, objects that turn or get hidden, arbitrary points — it is very general.
- **Its objective:** was trained to be right within a pixel or so on ordinary videos, not to be accurate to 1/100 of a pixel.
- **Weak spot here:** its noise is roughly constant *in the network's own pixels* ($\approx$0.15–0.2 px). The network just does not "see" finer than that.

Our trick to help it: zoom the crop 2$\times$ before giving it to the network, so 0.2 network-pixels becomes 0.1 real pixels. It helped, but 3$\times$ zoom made things worse (the network lost context). We also trained a small "refinement" network on top. Still, the final error stayed slightly above the simple template method.

Key idea: A general tool is not automatically the best tool for a very precise, very specific job.

Method C: Phase-based motion (Chen et al. 2015, Wadhwa et al. 2013)

Idea: instead of matching pictures, look at the *phase* of wave-like filters applied to the image. The phase moves *linearly* with the motion — so a tiny motion gives a tiny, clean, readable phase change.

- **Good at:** sub-pixel motion, very small vibrations, no need for a model to be trained. Used in "motion magnification" videos that make invisible vibrations visible.
- **Weak spot in the original papers:**
  - Designed for a **fixed camera** on a tripod. A drone is not a tripod.
  - Phase "wraps around" like a clock: after moving about 2.5 px the reading jumps back. Drone drift of ± 18 px breaks it completely.
  - Tested on high-speed, uncompressed video, not on compressed 4K from a consumer drone.

Key idea: This method has the *right sensitivity*. Our job is to make it survive a moving, compressed camera.

Summary: who can do what?

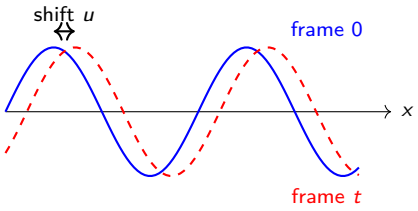
Method	sub-pixel precise?	handles big drift?	needs training?	our measured error
Template matching (ZNCC)	partly	yes	no	0.085 mm
CoTracker3 (deep learning)	no	yes	pre-trained	0.115 mm
CoTracker3 + refinement net	yes	yes	yes	0.076 mm
Local phase (original)	**yes**	**no**	no	(breaks: 0.162 mm)
Local phase (ours, adapted)	**yes**	**yes**	no	**0.075 mm**

All methods share the *same* steps after tracking, so the table compares only the tracker.

Part 3: The backbone — local phase

1. Part 1 — The Challenge
2. Part 2 — What people tried before
3. Part 3 — The backbone: local phase
4. Part 4 — Our adaptations (and the hypotheses behind them)
5. Part 5 — Experiments and results
6. Part 6 — Discussion and conclusion

What is “phase”? A wave analogy



- Imagine a wave drawn across the picture. Its **phase** tells you *where you are* on the wave (top, bottom, going up...).
- If the picture slides right by u pixels, the wave slides too, and the phase changes by a fixed amount:

$$\Delta\phi = -\phi_x\, u$$

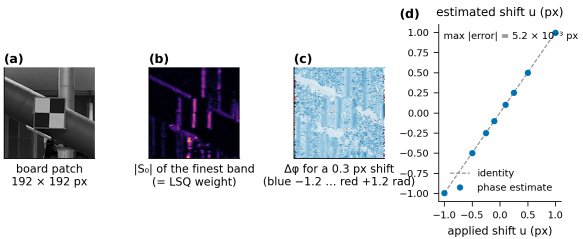
  where ϕ_x is how fast the phase changes per pixel (the wave's “frequency”).
- So: **measure $\Delta\phi$, divide by ϕ_x, and you get u** — even if u is 0.01 px!

Key idea: Phase turns “where is the edge?” (hard, pixel-limited) into “how much did the wave turn?” (easy, continuous).

Where do the waves come from? The steerable pyramid

A real picture is not a wave. But we can *filter* it into wave-like pieces:

- A **complex steerable pyramid** splits the picture into bands: 4 sizes (fine to coarse) $\times$ 2 directions (horizontal, vertical).
- Each band is a *complex* number at every pixel: an **amplitude** (how strong the pattern is) and a **phase** (where on the wave we are).
- Checkerboard edges produce strong, clean waves — that is why the target is a checkerboard!



(a) the board; (b) amplitude of one band — bright where the edges are; (c) phase change after a 0.3 px shift — the *same* value everywhere on the edges; (d) recovered shift vs. true shift: a straight line.

From many phases to one answer: least squares

Every pixel in every band gives one small equation (Chen et al., Eq. 3):

$$\phi_x \, u + \phi_y \, v + \Delta\phi = 0$$

Unknowns: the shift (u, v) — just **two numbers**. Equations: about **10,000** per frame.

- We solve them all at once by **weighted least squares**: pixels with strong amplitude (clear edges) count more; flat grey regions count almost nothing.
- Ten thousand noisy votes for two numbers $\Rightarrow$ a very stable answer.
- Bonus: the leftover disagreement between the votes tells us **how confident** we are in each frame (a predicted error σ).

Think about it: Why is averaging 10,000 measurements better than picking the single best one?

Part 4: Our adaptations

1. Part 1 — The Challenge
2. Part 2 — What people tried before
3. Part 3 — The backbone: local phase
4. Part 4 — Our adaptations (and the hypotheses behind them)
5. Part 5 — Experiments and results
6. Part 6 — Discussion and conclusion

Adaptation 1: Don't let the phase wrap — start from a coarse guess

Problem: phase is like a clock hand. Past ± 2.5 px on the finest band, it goes "around" and we cannot tell 3 px from 0.5 px. The drone drifts ± 18 px.
Hypothesis: *if we first get a rough position (to ~ 0.1 px) with simple template matching, then the phase only has to measure the small remainder — and never wraps.*
What we do:

1. Run ZNCC template matching first (rough track).
2. Cut the patch around the target at the **whole-pixel** part of that track (re-cut only when the integer changes).
3. Give the phase solver the **fraction** as a starting point; it corrects the last ~ 0.1 px.
4. One extra "Gauss–Newton" pass: shift the reference by the current estimate and solve once more, removing a tiny bias.

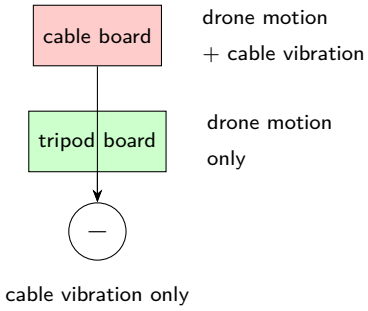
Test of the hypothesis: without the coarse guess, 44% of frames wrap and the error **doubles** (0.162 mm). With it: 0 frames wrap, 0.075 mm. Hypothesis confirmed.

Adaptation 2: Cancel the drone's motion with a still reference

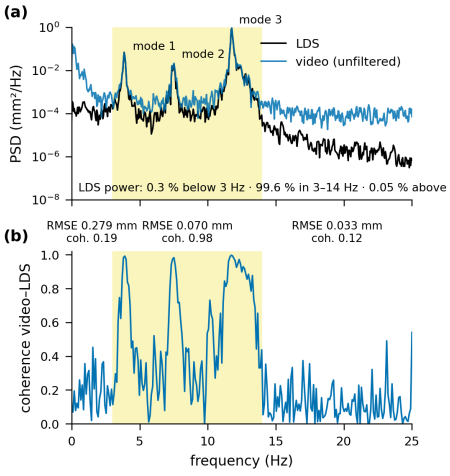
Problem: the whole picture moves because the drone moves. We only want the cable's motion.
Hypothesis: *a second checkerboard that is standing still, at the same distance from the camera, moves in the picture only because of the drone. Subtract its track and the drone disappears.*
What we do: track both boards with the same method, then

$$\text{cable motion} = \text{cable track} - \text{tripod track}.$$

Then rotate the result onto the direction perpendicular to the cable (26.5°, measured from the picture) and convert px → mm (27.5 mm square = 31.09 px).
Test: without subtraction, error = 0.249 mm; with it, 0.085 mm (ZNCC). A 3× improvement.



Adaptation 3: Keep only the frequencies where the cable really moves

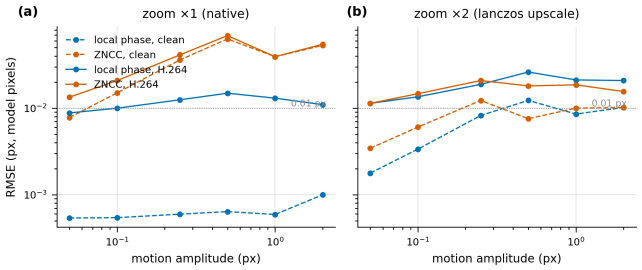


Problem: after subtraction a slow wobble remains. The two boards are not at *exactly* the same distance, so the drone's motion is not cancelled perfectly (this leftover is called **parallax**).

Hypothesis: *the cable vibrates only at certain "notes" (3.9, 7.5, 11.7 Hz). The leftover wobble is slow (<3 Hz). A band-pass filter 3–14 Hz keeps the music and removes the wobble.*

Test (figure): (a) 99.6% of the laser's energy is inside 3–14 Hz. (b) Video and laser **agree** (coherence 0.98) inside the band and **disagree** (0.19) below 3 Hz — so what is below 3 Hz is not real cable motion. Filtering cuts the error from 0.29 mm to 0.075 mm.

Adaptation 4 (a small one): run at native resolution, not zoomed

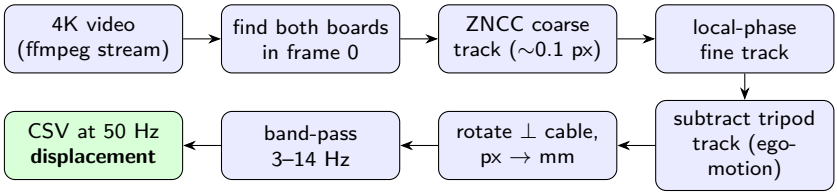


Question: zooming 2× helped CoTracker3. Does it help the phase tracker too?
Test on synthetic video (we move the real board by a known amount, so we know the true answer):

- Clean video: phase error ≈ 0.001 px (!)
- After H.264 compression: 0.009–0.015 px for phase, 0.04–0.07 px for ZNCC.
- Zooming helps ZNCC but **hurts** phase.

Conclusion: phase runs on the original pixels. The compression, not our maths, sets the floor.

The whole pipeline, step by step



- Everything is plain NumPy/SciPy; no training, no random numbers ⇒ the same answer every time.
- Runtime for 3033 frames: **29 s on one GPU**, about 5 min on 8 CPU cores.
- To evaluate, we add: align to the laser by cross-correlation (offset found: 0.20 s), remove means, compute RMSE.

Part 5: Experiments and results

1. Part 1 — The Challenge
2. Part 2 — What people tried before
3. Part 3 — The backbone: local phase
4. Part 4 — Our adaptations (and the hypotheses behind them)
5. Part 5 — Experiments and results
6. Part 6 — Discussion and conclusion

Experimental setup

Data the competition video (60.66 s, 3033 frames, 50 fps, H.264) and the laser file (10 kHz).

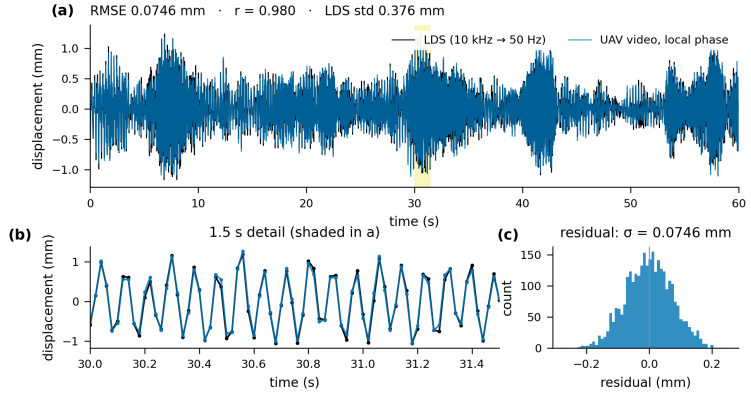
Reference laser signal, low-pass filtered and resampled to 50 Hz; mean removed; aligned by cross-correlation.

Metric RMSE (mm) and correlation r in the 3–14 Hz band, over 3000 overlapping samples.

Fair comparison all five trackers use the *same* detection, ego-motion cancellation, rotation and filter — only the tracker changes.

Extra checks a synthetic benchmark with known truth; 14 variants of the phase tracker ("ablations"); a try at a fancier ego-motion model; and a visual check with motion magnification.

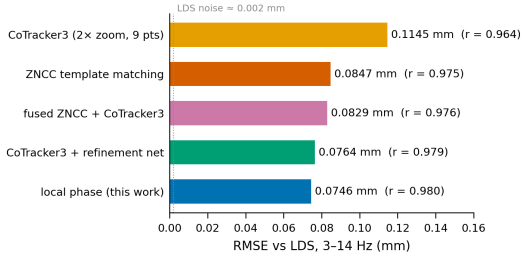
Main result: video vs. laser over the full minute



(a) black = laser, blue = our video result — they lie on top of each other. (b) a 1.5 s zoom. (c) the leftover error is small and bell-shaped.

RMSE = 0.0746 **mm,** $r = 0.980$**.** The vibration's own size is 0.38 mm, so the error is about 1/5 of the signal — and only **0.084 px**.

Comparing the five trackers



tracker	RMSE (mm)	r
CoTracker3 (2× zoom)	0.1145	0.964
ZNCC template matching	0.0847	0.975
0.8 ZNCC + 0.2 CoTracker3	0.0829	0.976
CoTracker3 + refinement CNN	0.0764	0.979
Local phase (ours)	**0.0746**	**0.980**

- The phase tracker wins — with **no training** and the shortest runtime.
- The trained CNN gets close, but needs minutes of training and changes slightly each time it is retrained.
- Notice: the differences are small (0.075 vs 0.085). Something *shared* by all methods seems to limit them. . .

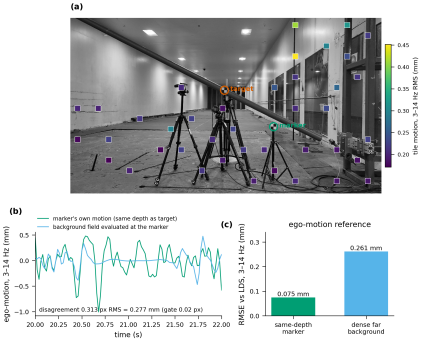
Ablations: which parts of the tracker matter?

We changed one thing at a time and re-ran the whole video (14 variants):

variant	RMSE 3–14 Hz (mm)	tracker's own noise σ (mm)
baseline (4 scales, 2 orientations)	0.0746	0.0085
only the finest scale	0.0758	0.0146
4 orientations instead of 2	0.0743	0.0073
no Gauss–Newton pass	0.0746	0.0086
coarser (decimate 4, as in the original paper)	0.0842	0.0491
no ZNCC coarse guess	**0.1617**	0.0119

- Almost every change lands within ± 0.001 mm of the baseline — **even when the tracker's own noise changes 6×**.
- The one thing that really matters: the coarse guess (Adaptation 1).
- **Reading:** the tracker noise ($\sim$0.008 mm) is **ten times smaller** than the error (0.075 mm). So the error is *not* coming from the tracker any more.

A “good idea” that failed: dense background ego-motion



Hypothesis: *one tripod board is a single noisy reference. Tracking 40 background patches and fitting a smooth motion field should cancel the drone better.*

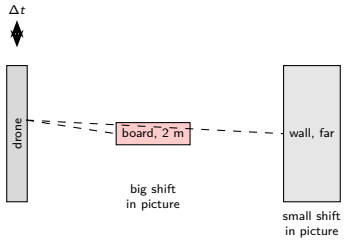
Result: the error got **worse**: 0.075 → 0.261 mm. (b) shows the field and the tripod board disagreeing by 0.31 px.

Why did it fail? Parallax — the “thumb trick”

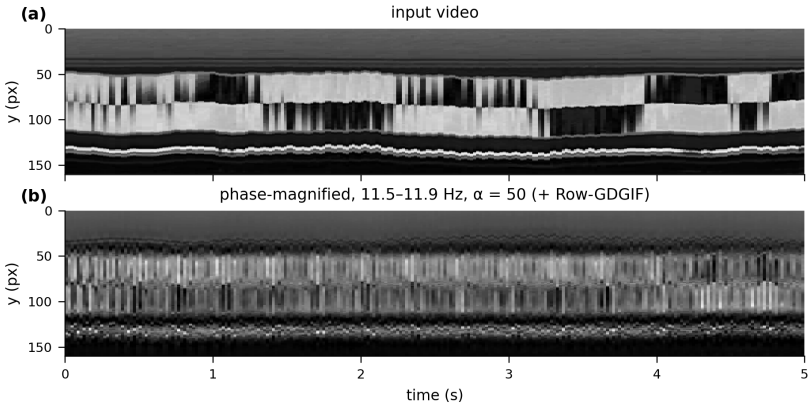
Hold your thumb in front of your face and move your head sideways. Your thumb jumps a lot against the wall; the wall barely moves.

- Near things move more in the picture than far things when the camera moves: $\Delta x = f\,\Delta t / Z$ (shift $\propto 1/\text{distance}$).
- The tripod board is at 2 m, **like the cable**. The wall behind is several metres away.
- So the wall's motion is *not* the same as the cable's would be. Using it as a reference **adds** error.

Key idea: A reference must be at the *same distance* as the target. More data from the wrong distance is worse than less data from the right one.



Visual check: making the invisible visible



We used phase-based **motion magnification** (50×, only around 11.7 Hz) to *see* the third vibration mode, which is only 0.02 px big. The whole board moves up and down as one rigid piece — so it is real cable motion, not a tracker artefact. (Used only for checking, never for measuring.)

Part 6: Discussion and conclusion

1. Part 1 — The Challenge
2. Part 2 — What people tried before
3. Part 3 — The backbone: local phase
4. Part 4 — Our adaptations (and the hypotheses behind them)
5. Part 5 — Experiments and results
6. Part 6 — Discussion and conclusion

Discussion: where does the remaining 0.075 mm come from?

- **Not the tracker:** its own noise is $\sim$0.008 mm, and 13 very different variants all give the same error.
- **Not the laser:** its noise in our band is 0.002 mm.
- **Most likely the scene:**
  - the two boards are at *slightly* different distances, so a little parallax leaks into the 3–14 Hz band;
  - the tripod itself may vibrate a tiny bit;
  - the camera reads the picture line by line (**rolling shutter**), so the two boards are captured at slightly different moments. We tried to estimate this and got two inconsistent answers, so we did not correct it.
- With **one camera**, we cannot separate these. A second still board right next to the cable would help.

Think about it: If you could add one object to the scene to improve the measurement, what would it be and where would you put it?

Honest limitations

- The 3–14 Hz filter is chosen for *this* cable. A different structure needs its own band (chosen the same way: look at the spectrum and the coherence).
- Needs a still reference object at the target's distance. Without it, the drone motion cannot be removed.
- Needs a textured target (edges). A plain surface has no "waves" for the phase to read.
- Motion larger than $\sim$2 px between frames needs the coarse guess to be good — for very fast motion a faster camera would be needed.

Conclusion

1. We measured a ± 1.5 px vibration from a hovering drone to **0.075 mm RMSE ($r = 0.98$)** over a full minute — the best of five trackers, with no training and 29 s of compute.
2. The backbone is **local phase** from a steerable pyramid (Chen et al. 2015): a linear, sub-pixel ruler.
3. Three adaptations made it work on a drone: **(1)** a coarse ZNCC guess so phase never wraps, **(2)** a same-depth reference board to cancel drone motion, **(3)** a 3–14 Hz band chosen from the data's own spectrum.
4. Each adaptation was tested as a hypothesis, and one attractive idea (dense background field) was tested and **rejected** — because of parallax.
5. The tracker is now ten times more precise than the final error; the remaining error belongs to the scene, not to the maths.

Code: `python inference/measure_displacement.py Video.MP4 -out displacement.csv`

References and open-source tools

- Chen, J.G., Wadhwa, N., Cha, Y.-J., Durand, F., Freeman, W.T., Buyukozturk, O. *Modal identification of simple structures with high-speed video using motion magnification.* J. Sound Vib. 345 (2015) 58–71.
- Wadhwa, N., Rubinstein, M., Durand, F., Freeman, W.T. *Phase-based video motion processing.* ACM Trans. Graph. 32(4), 2013. Simoncelli & Freeman (1995), the steerable pyramid.
- Yang, Y., Jiang, S. *Phase-based video motion magnification with an optimized 1D row guided dynamic gradient image filter.* MSSP 215 (2024) — used for the visual check only.
- Karaev, N. et al. *CoTracker3.* arXiv:2410.11831 (2024) — baseline tracker, public checkpoint via torch.hub.
- FFmpeg, NumPy, SciPy, Matplotlib, PyTorch.
- Data: competition video and LDS file (Harbin Institute of Technology wind-tunnel benchmark). No other data or models enter the submitted result.

Thank you! Questions?